

Manual — FLASH-RF Configuration Performance Tuning Tool

1 Overview

FLASH-RF is a software configuration performance tuning toolkit based on Random Forest sequential model optimisation. It includes seven iterative versions (six forward iterations + two experimentally rejected candidates):

Version	Script	Algorithm	Status
v1	<code>flash_tuner.py</code>	RF(10) + mean ranking	Baseline
v2	<code>flash_tuner_v2.py</code>	RF(20) + LCB + feature preprocessing	Comparison
v3	<code>flash_tuner_v3.py</code>	LightGBM ensemble + LCB	Experimental (rejected)
v4	<code>flash_tuner_v4.py</code>	RF(20) + Thompson Sampling	Comparison
v5	<code>flash_tuner_v5.py</code>	v4 + conditional log(1+y) transform	Comparison
v6	<code>flash_tuner_v6.py</code>	v5 + conditional stratified initial sampling	Recommended (final)
v7	<code>flash_tuner_v7.py</code>	v6 + conditional rank transform	Experimental (rejected)

Recommended: v6 — significantly better than RS on 8/8 systems, significantly better than v5 on 7z ($p < 0.05$), zero degradation. v7 attempted rank transform in place of log transform but was rejected due to degradation on 3 systems.

2 Quick Start

1. Install dependencies

```
pip install -r requirements.txt
```

2. Run the recommended version v6 (budget=100, 30 repeats)

```
PYTHONPATH=iteration1:iteration2:iteration4:iteration5 \  
python iteration6/flash_tuner_v6.py --datasets datasets/
```

3. Results are saved to iteration6/results/ (relative to project root)

Note (cross-iteration imports): v2–v7 import modules from earlier iterations. When running from the project root, set PYTHONPATH as shown above. v1 and `run_ablation.py` are self-contained and need no prefix.

3 Command-Line Arguments

All versions share the same argument interface:

Argument	Default	Description
<code>--datasets</code>	<code>datasets</code>	Path to directory containing CSV dataset files
<code>--budget</code>	<code>100</code>	Measurement budget per run
<code>--repeats</code>	<code>30</code>	Number of independent runs per algorithm

Argument	Default	Description
<code>--output</code>	<code>results (v1) / results_v{n} (v2-v7)</code>	Output directory for results

3.1 Supplementary experiment scripts

Script	Argument	Default	Description
<code>flash_tuner_v2.py</code>	<code>--no-tpe</code>	—	Skip TPE comparison (faster)
<code>run_ablation.py</code>	<code>--skip-tpe</code>	—	Skip TPE comparison (run sensitivity only)
(continued)	<code>--skip-sensitivity</code>	—	Skip sensitivity analysis (run TPE only)
<code>run_extended_experiments.py</code>	<code>--datasets</code>	<code>datasets</code>	Dataset directory
	<code>--output</code>	<code>results_extended</code>	Output directory
<code>compare_budgets.py</code>	—	—	No CLI arguments; hardcoded budgets=[30,100], compares v5 across budgets
<code>flash_tuner_v5_r30.py</code>	<code>--output</code>	<code>results_v5_r30</code>	Small-budget variant of v5 (init_ratio=0.50 for budget≤30)
<code>analyze_v5_vs_v4.py</code>	—	—	No CLI arguments; prints v5 vs v4 degradation analysis to stdout

4 Input Data Format

Each CSV file in the datasets directory should have the following structure: - **Columns 1 to n-1**: Configuration parameters (integer values) - **Column n (last)**: Performance metric to be minimised

5 Usage Examples and Output

```
# Define a PYTHONPATH shorthand (v2-v7 depend on earlier iterations)
export PY="PYTHONPATH=iteration1:iteration2:iteration4:iteration5:iteration6"
```

5.1 v1 baseline (flash_tuner.py)

```
python iteration1/flash_tuner.py --datasets datasets/
```

Output:

```
iteration1/results/
├── summary.csv      # Summary: RS / v1 comparison
├── results.json     # Full results with raw data
└── raw/             # Per-system raw results (30 runs)
```

5.2 Supplementary experiments (run_ablation.py)

```
python iteration1/run_ablation.py --datasets datasets/
python iteration1/run_ablation.py --skip-sensitivity # TPE only
python iteration1/run_ablation.py --skip-tpe        # sensitivity only
```

Output:

```
iteration1/results/ablation/
├── tpe_comparison.csv # TPE vs FLASH-RF vs RS three-way summary
└── tpe_comparison.json # Same data in JSON format
```

```
└─ tpe_raw/                                # Per-system three-way raw results
└─ sensitivity_init_ratio.csv              # Hyperparameter sensitivity analysis
└─ sensitivity_batch_size.csv
└─ sensitivity_exploit_ratio.csv
└─ ablation_report.md                     # Auto-generated ablation report
```

5.3 v2 (flash_tuner_v2.py — RF + LCB + feature preprocessing)

```
PYTHONPATH=iteration1 \
python iteration2/flash_tuner_v2.py --datasets datasets/
```

Output:

```
iteration2/results/
└─ summary.csv                # Summary: RS / v1 / v2 / TPE comparison
└─ results.json
└─ raw/
```

5.4 v3 (flash_tuner_v3.py — LightGBM, rejected)

Requires: `pip install lightgbm`

```
PYTHONPATH=iteration1:iteration2 \
python iteration3/flash_tuner_v3.py --datasets datasets/
```

Output:

```
iteration3/results/
└─ summary.csv                # Summary: RS / v2 / v3-LGB comparison
└─ results.json
└─ raw/
```

5.5 v4 (flash_tuner_v4.py — RF + Thompson Sampling)

```
$PY python iteration4/flash_tuner_v4.py --datasets datasets/
```

Output:

```
iteration4/results/
└─ summary.csv                # Summary: RS / v2 / v4 comparison
└─ results.json
└─ raw/
```

5.6 v5 (flash_tuner_v5.py — RF + TS + conditional log transform)

```
$PY python iteration5/flash_tuner_v5.py --datasets datasets/
```

Output:

```
iteration5/results/
└─ summary.csv                # Summary: RS / v4 / v5 comparison
└─ results.json
└─ raw/
```

5.7 v5 small-budget variant (flash_tuner_v5_r30.py)

```
$PY python iteration5/flash_tuner_v5_r30.py --datasets datasets/
```

Output:

```
iteration5/results_r30/
├── summary.csv
├── results.json
└── raw/
```

5.8 v5 vs v4 degradation analysis (analyze_v5_vs_v4.py)

```
python iteration5/analyze_v5_vs_v4.py
```

Prints per-system v5 vs v4 comparison to stdout (no file output).

5.9 v6 — recommended final version (flash_tuner_v6.py)

```
$PY python iteration6/flash_tuner_v6.py --datasets datasets/
```

Custom budget and repeats

```
$PY python iteration6/flash_tuner_v6.py \
    --datasets datasets/ --budget 200 --repeats 50
```

Output:

```
iteration6/results/
├── summary.csv          # Summary: RS / v5 / v6 comparison
├── results.json        # Full results with raw data
└── raw/
    ├── 7z.csv          # Per-system raw results (30 runs)
    ├── Apache.csv
    └── ...
```

5.10 v7 (flash_tuner_v7.py — rank transform, rejected)

```
$PY python iteration7/flash_tuner_v7.py --datasets datasets/
```

Output:

```
iteration7/results_v7/
├── summary.csv          # Summary: RS / v6 / v7 comparison
├── results.json
└── raw/
```

5.11 Extended experiments (run_extended_experiments.py)

```
$PY python iteration6/run_extended_experiments.py \
    --datasets datasets/
```

Multi-budget comparison

```
$PY python iteration6/compare_budgets.py
```

Output:

```
iteration6/results_extended/
├── results_extended.json # Full results (all methods x systems x budgets)
├── budget_30/
│   ├── summary.csv
│   └── raw/
└── budget_100/
```

```

├── summary.csv
├── raw/
└── budget_200/
    ├── summary.csv
    └── raw/

```

5.12 Report figures (generate_figures.py)

python docs/generate_figures.py

Output:

```

docs/figures/
├── fig1_per_system_hit.png      # Per-system hit-optimal comparison (R=100)
├── fig2_total_hit.png          # Aggregate hit-optimal (R=100)
├── fig3_multi_budget.png       # Multi-budget robustness (R=30/100/200)
└── fig4_iteration_evolution.png # Iteration evolution (significance trends)

```

6 Interpreting Results

- **$\Delta\%$:** Positive values indicate the method outperforms Random Search (median improvement percentage).
- **p-value:** From Mann–Whitney U test (one-sided). Values < 0.05 indicate statistically significant improvement.
- **Significance markers:** *** ($p < 0.001$), ** ($p < 0.01$), * ($p < 0.05$), ns (not significant).
- **Hit-optimal:** Number of times the global optimum configuration was found out of 30 independent runs.

7 Version Differences

v1 → v2: Fix feature encoding defects + introduce LCB uncertainty guidance
v2 → v3: LightGBM replaces RF (rejected – did not outperform v2)
v2 → v4: Thompson Sampling replaces LCB
(3/8 systems significantly better than v2)
v4 → v5: Conditional log(1+y) transform
(zero degradation, marginal improvement on skewed systems)
v5 → v6: Conditional stratified initial sampling
(7z significantly improved, 8/8 all significantly better than RS)
v6 → v7: Conditional rank transform replaces log
(rejected – 3 systems degraded)

v6 finality confirmation:

Optimisation dimension	Version tested	Conclusion
Surrogate model	v3 (LightGBM)	RF is optimal for $n \leq 100$; Boosting cannot outperform Bagging
Acquisition strategy	v4 (Thompson Sampling)	TS eliminates LCB's κ hyperparameter, 3/8 significant improvements
Label data transform	v5 (conditional log) / v7 (conditional rank)	log(1+y) is optimal; rank causes degradation

Optimisation dimension	Version tested	Conclusion
Unlabelled data utilisation	v6 (conditional stratified sampling)	X-space structure guides Phase-1, 7z significantly improved

8 Dependencies

Package	Version	Purpose	Required
numpy	≥ 1.24	Numerical computation	Yes
pandas	≥ 2.0	Data loading and output	Yes
scikit-learn	≥ 1.3	RandomForestRegressor, MiniBatchKMeans	Yes
scipy	≥ 1.11	Mann-Whitney U test, rankdata	Yes
optuna	≥ 3.0	TPE comparison experiments	Optional
lightgbm	≥ 4.0	v3 LightGBM experiment	Optional
matplotlib	≥ 3.7	Report figure generation	Optional